cred-platform-simplified

File & Function Reference

Generated 2026-03-04 02:42:42

What this repo does

This monorepo runs three small FastAPI services on one machine (Docker Compose): **cert_service**: converts uploaded PFX/P12 to PEM files, issues a provider token internally, then stores both into configuration_service. **configuration_service**: persists tokens + converted PEM files to disk under a generated config_id. **backend_api** (optional): thin wrapper that forwards uploads to cert_service and returns config_id (+ token). Provider logic is **inside** cert_service (no separate provider microservice).

File index

Path	Purpose
.env.example	Sample environment variables for running the stack locally via Docker Compose.
README.md	Project overview, quickstart instructions, and example curl commands.
docker-compose.yml	Defines the local multi-container stack (backend_api, cert_service, configuration_service), ports, env wiring, and volume mounts.
services/backend_api/Dockerfile	Docker image build for backend_api (installs requirements and runs uvicorn).
services/backend_api/app/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/backend_api/app/api/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/backend_api/app/api/routes.py	backend_api endpoints: health + request-token wrapper that calls cert_service and (if needed) configuration_service.
services/backend_api/app/api/schemas.py	Pydantic models for backend_api responses.
services/backend_api/app/core/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/backend_api/app/core/config.py	Pydantic Settings for backend_api (service API key, URLs, timeouts) and cached loader.
services/backend_api/app/core/logging.py	Basic logging configuration helper (stdout format + httpx noise reduction).
services/backend_api/app/main.py	FastAPI app factory for backend_api; configures logging and mounts the API router.
services/backend_api/app/services/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/backend_api/requirements.txt	Python dependencies for backend_api.
services/cert_service/Dockerfile	Docker image build for cert_service (installs requirements and runs uvicorn).
services/cert_service/app/__init__.py	Empty package marker so Python treats this directory as a module/package.

Path	Purpose
services/cert_service/app/api/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/cert_service/app/api/routes.py	cert_service endpoints: health + convert-store flow (convert -> get token -> store config).
services/cert_service/app/api/schemas.py	Pydantic models for cert_service responses (token + config ID/location).
services/cert_service/app/core/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/cert_service/app/core/config.py	Pydantic Settings for cert_service (conversion mode, provider token overrides, config-service URL) and cached loader.
services/cert_service/app/core/logging.py	Basic logging configuration helper.
services/cert_service/app/main.py	FastAPI app factory for cert_service; configures logging and mounts the API router.
services/cert_service/app/services/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/cert_service/app/services/certs.py	PFX/P12 to PEM conversion logic (library or CLI) plus light parsing to detect key/certs.
services/cert_service/app/services/config_store.py	HTTP client helper to store converted certs + token into configuration_service.
services/cert_service/app/services/providers.py	Internal provider token stubs (openai/auth) used by cert_service (no separate provider microservice).
services/cert_service/requirements.txt	Python dependencies for cert_service (includes cert-convert).
services/configuration_service/Dockerfile	Docker image build for configuration_service (installs requirements and runs uvicorn).
services/configuration_service/app/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/configuration_service/app/api/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/configuration_service/app/api/routes.py	configuration_service endpoints: health, create config, get config (optionally include files).
services/configuration_service/app/api/schemas.py	Pydantic models for create/get config payloads stored by configuration_service.
services/configuration_service/app/core/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/configuration_service/app/core/config.py	Pydantic Settings for configuration_service (storage dir, optional service API key) and cached loader.
services/configuration_service/app/core/logging.py	Basic logging configuration helper.
services/configuration_service/app/main.py	FastAPI app factory for configuration_service; configures logging and mounts the API router.
services/configuration_service/app/services/__init__.py	Empty package marker so Python treats this directory as a module/package.
services/configuration_service/app/services/storage.py	Disk-backed storage layer: writes cert files and a manifest.json; reads them back (optional file inclusion).
services/configuration_service/requirements.txt	Python dependencies for configuration_service.

Python modules: definitions and behavior

For each Python file, this section lists its top-level functions/classes and (for classes) the methods defined in that file.

Root files

Service: backend_api

services/backend_api/app/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/backend_api/app/api/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/backend_api/app/api/routes.py

backend_api endpoints: health + request-token wrapper that calls cert_service and (if needed) configuration_service.

Kind	Name	What it does
function	_auth	Validates the optional X-Service-API-Key header against the configured SERVICE_API_KEY.
function	health	Health check endpoint; returns {'status':'ok'} after optional API-key validation.
function	request_token	Accepts a PFX upload, forwards it to cert_service /convert-store/{provider}, and returns config_id plus token. If cert_service doesn't return the token (config option), it fetches it from configuration_service.

services/backend_api/app/api/schemas.py

Pydantic models for backend_api responses.

Kind	Name	What it does
class	BackendResponse	Response model returned by backend_api; includes provider, config_id, optional token, and placeholder llm_invoked flag.

services/backend_api/app/core/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/backend_api/app/core/config.py

Pydantic Settings for backend_api (service API key, URLs, timeouts) and cached loader.

Kind	Name	What it does
function	get_settings	Loads Settings from environment once and caches them (lru_cache).
class	Settings	Pydantic Settings for backend_api: app name/log level, optional service API key, URLs to cert_service & configuration_service, and HTTP timeout.

services/backend_api/app/core/logging.py

Basic logging configuration helper (stdout format + httpx noise reduction).

Kind	Name	What it does
function	configure_logging	Sets logging level/format and quiets chatty http client logs.

services/backend_api/app/main.py

FastAPI app factory for backend_api; configures logging and mounts the API router.

Kind	Name	What it does
function	create_app	Creates the FastAPI app, loads settings, configures logging, and mounts the router under /v1.

services/backend_api/app/services/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

Service: cert_service

services/cert_service/app/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/cert_service/app/api/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/cert_service/app/api/routes.py

cert_service endpoints: health + convert-store flow (convert -> get token -> store config).

Kind	Name	What it does
function	_auth	Validates the optional X-Service-API-Key header against the configured SERVICE_API_KEY.
function	health	Health check endpoint for cert_service.
function	convert_store	Main endpoint: parses inputs, converts PFX to PEMs, issues a provider token (internal), stores token+PEMs in configuration_service, returns config_id/location and optional token.

services/cert_service/app/api/schemas.py

Pydantic models for cert_service responses (token + config ID/location).

Kind	Name	What it does
class	TokenResult	Token model (token string + optional type/expiry) used in responses and for storage.
class	ConvertStoreResponse	Response from cert_service /convert-store: provider + config id/location + optional token + light 'detected' info.

services/cert_service/app/core/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/cert_service/app/core/config.py

Pydantic Settings for cert_service (conversion mode, provider token overrides, config-service URL) and cached loader.

Kind	Name	What it does
function	get_settings	Loads Settings from environment once and caches them (lru_cache).
class	Settings	Pydantic Settings for cert_service: cert-convert integration options, config-service URL, optional service API key, and provider token overrides.

services/cert_service/app/core/logging.py

Basic logging configuration helper.

Kind	Name	What it does
function	configure_logging	Sets logging level/format and quiets http client logs.

services/cert_service/app/main.py

FastAPI app factory for cert_service; configures logging and mounts the API router.

Kind	Name	What it does
function	create_app	Creates the FastAPI app, loads settings, configures logging, and mounts the router under /v1.

services/cert_service/app/services/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/cert_service/app/services/certs.py

PFX/P12 to PEM conversion logic (library or CLI) plus light parsing to detect key/certs.

Kind	Name	What it does
function	convert_pfx	Writes the uploaded PFX/P12 to a temp folder, runs conversion (library or CLI), reads PEM outputs, detects key+certs, and returns a CertBundle.
function	_convert_library	Calls your cert-convert Python callable to perform the conversion.
function	_resolve	Finds a usable cert-convert callable from CERT_CONVERT_CALLABLE or common defaults.
function	_load	Imports a 'module:callable' reference and returns the callable object.
function	_convert_cli	Runs the configured cert conversion CLI template (e.g., certs-tool ...) as a subprocess.
function	_detect	Heuristically detects the private key PEM and certificate PEM blocks from the generated files.
class	CertBundle	Dataclass that holds all generated PEM files plus detected private key and certificates.

services/cert_service/app/services/config_store.py

HTTP client helper to store converted certs + token into configuration_service.

Kind	Name	What it does
function	store	Base64-encodes cert files and POSTs {provider, token, files, metadata} to configuration_service; returns the stored config response.

services/cert_service/app/services/providers.py

Internal provider token stubs (openai/auth) used by cert_service (no separate provider microservice).

Kind	Name	What it does
function	issue_token	Internal provider stub. Returns a token for 'openai' or 'auth' using env override (OPENAI_STATIC_TOKEN/AUTH_STATIC_TOKEN) or generates a UUID token.

Service: configuration_service

services/configuration_service/app/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/configuration_service/app/api/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/configuration_service/app/api/routes.py

configuration_service endpoints: health, create config, get config (optionally include files).

Kind	Name	What it does
function	_auth	Validates the optional X-Service-API-Key header against the configured SERVICE_API_KEY.
function	health	Health check endpoint for configuration_service.
function	create_config	Stores a config: writes PEM files + manifest to disk and returns the new config_id and location.
function	get_config	Loads a stored config by id. Optionally includes base64-encoded cert files if include_files=true.

services/configuration_service/app/api/schemas.py

Pydantic models for create/get config payloads stored by configuration_service.

Kind	Name	What it does
class	TokenResult	Token model stored in the configuration service.
class	CreateConfigRequest	Request payload to store a config: provider, token, base64 cert files, and optional metadata/raw fields.
class	CreateConfigResponse	Response after storing a config: config_id and on-disk location.
class	GetConfigResponse	Response when fetching a config: includes token and optionally base64 cert files.

services/configuration_service/app/core/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

services/configuration_service/app/core/config.py

Pydantic Settings for configuration_service (storage dir, optional service API key) and cached loader.

Kind	Name	What it does
function	get_settings	Loads Settings from environment once and caches them (lru_cache).
class	Settings	Pydantic Settings for configuration_service: storage directory, optional service API key, and logging config.

services/configuration_service/app/core/logging.py

Basic logging configuration helper.

Kind	Name	What it does
function	configure_logging	Sets logging level/format and quiets http client logs.

services/configuration_service/app/main.py

FastAPI app factory for configuration_service; configures logging and mounts the API router.

Kind	Name	What it does
function	create_app	Creates the FastAPI app, loads settings, configures logging, and mounts the router under /v1.

services/configuration_service/app/services/__init__.py

Empty package marker so Python treats this directory as a module/package.

No functions/classes defined in this file.

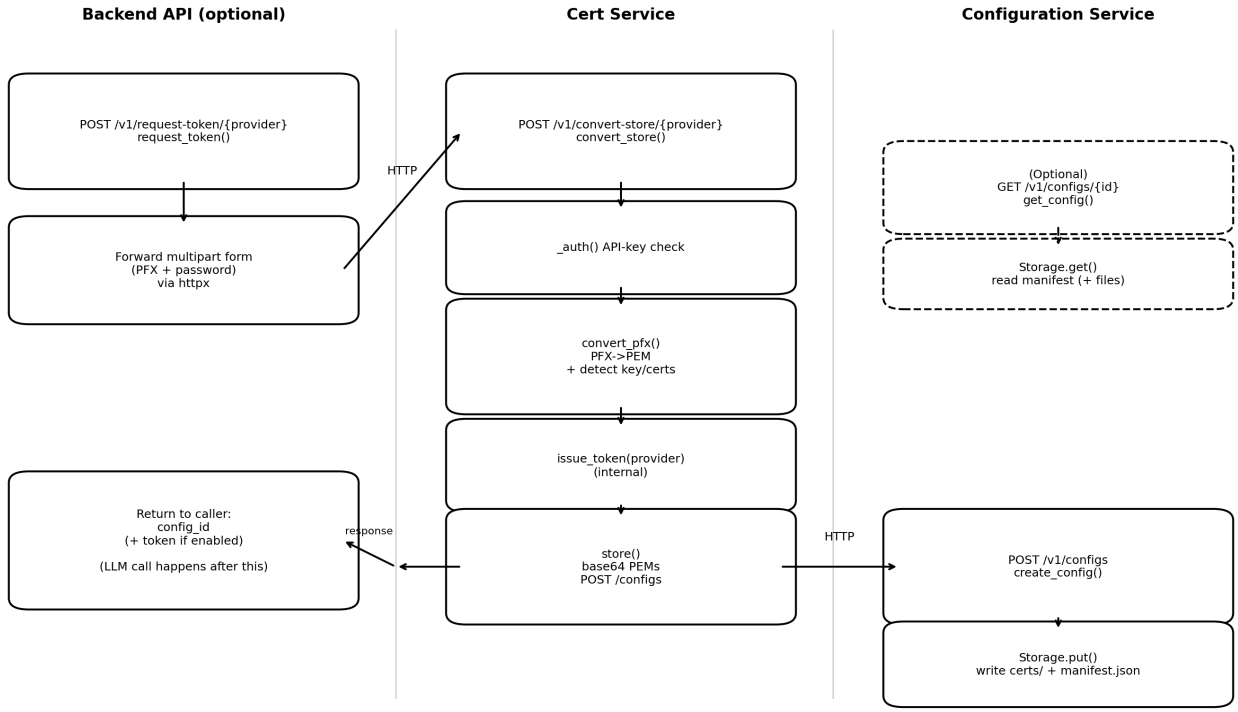
services/configuration_service/app/services/storage.py

Disk-backed storage layer: writes cert files and a manifest.json; reads them back (optional file inclusion).

Kind	Name	What it does
class	StoredConfig	Small dataclass returned by Storage.put, carrying the generated config_id and its on-disk directory.
class	Storage	Disk-backed storage utility used by configuration_service.
method	Storage.__init__	Initializes the storage root directory (creates it if needed).
method	Storage.put	Creates a new config folder, writes cert files under certs/, and writes manifest.json with token+metadata.
method	Storage.get	Reads manifest.json for a config. If include_files is true, base64-encodes and returns all PEM files.

Code flow diagram

This diagram summarizes the main runtime call chain across the services.



Dashed boxes represent an optional retrieval path (GET /v1/configs/{id}) used when a caller wants to fetch stored data later.